

1 Полная математическая постановка задачи

Рассматривается задача восстановления нейтронного спектра по откликам детекторов. Для каждого образца заданы:

$$\mathbf{x} = (x_1, \dots, x_n)^\top \in \mathbb{R}^n, \quad \mathbf{y} = (y_1, \dots, y_m)^\top \in \mathbb{R}^m, \quad (1)$$

где в основной постановке $n = 10$ — число каналов детекторов $Q_1, \dots, Q_{10}$, а $m = 60$ — число энергетических бинов спектра.

Требуется построить отображение

$$\hat{\mathbf{y}} = f_\theta(\mathbf{x}), \quad (2)$$

где f_θ — многовыходная ANFIS-модель с параметрами θ .

В текущей основной версии используется двухэтапная схема обучения:

1. построение базовой модели ANFIS на данных, указанных в `dataset.train_data`;
2. дифференцируемая донастройка той же модели на реальных измерениях с SHAP-регуляризацией и тихоновской регуляризацией гладкости спектра.

Ниже приведены именно те формулы, которые соответствуют текущей реализации в проекте `bonner-shap-reconstruction` для основной версии `Two-Stage SHAP + Tikhonov`.

2 Нормализация по сумме сигналов

В основном конфиге включена нормализация по сумме входных каналов (`normalize_sum=true`). Для i -го образца вводится

$$S_i = \sum_{j=1}^n x_{ij}. \quad (3)$$

Если $S_i = 0$, то в коде он заменяется малой положительной величиной $\varepsilon_{\text{sum}} > 0$.

Нормализованные входы и выходы имеют вид

$$\mathbf{x}'_i = \frac{\mathbf{x}_i}{S_i}, \quad \mathbf{y}'_i = \frac{\mathbf{y}_i}{S_i}. \quad (4)$$

Обучение выполняется в нормализованном пространстве, а при необходимости денормализация прогноза производится по формуле

$$\hat{\mathbf{y}}_i = S_i \hat{\mathbf{y}}'_i. \quad (5)$$

Такая схема сохраняет форму спектра и убирает доминирование образцов с большой абсолютной интенсивностью.

3 Многовыходная ANFIS-модель

Используется Sugeno-type ANFIS с R правилами. В основной конфигурации $R = 50$, а тип функций принадлежности — Gaussian.

Для правила $r \in \{1, \dots, R\}$ задаются посылки

$$\text{IF } x_1 \text{ is } A_{r1} \text{ AND } \dots \text{ AND } x_n \text{ is } A_{rn}, \quad (6)$$

и линейное многовыходное следствие

$$\mathbf{z}_r(\mathbf{x}) = A_r \mathbf{x} + \mathbf{b}_r, \quad A_r \in \mathbb{R}^{m \times n}, \quad \mathbf{b}_r \in \mathbb{R}^m. \quad (7)$$

Для Gaussian membership function можно записать

$$\mu_{rj}(x_j) = \exp\left(-\frac{(x_j - c_{rj})^2}{2\sigma_{rj}^2 + \varepsilon}\right), \quad (8)$$

где c_{rj} и σ_{rj} — центр и ширина функции принадлежности.

Сила срабатывания правила вычисляется как произведение принадлежностей:

$$w_r(\mathbf{x}) = \prod_{j=1}^n \mu_{rj}(x_j). \quad (9)$$

Нормализованный вес правила:

$$\bar{w}_r(\mathbf{x}) = \frac{w_r(\mathbf{x})}{\sum_{s=1}^R w_s(\mathbf{x}) + \varepsilon}. \quad (10)$$

Тогда итоговый прогноз по всем m выходам равен

$$\hat{\mathbf{y}} = f_{\theta}(\mathbf{x}) = \sum_{r=1}^R \bar{w}_r(\mathbf{x}) \mathbf{z}_r(\mathbf{x}). \quad (11)$$

Таким образом, каждый выходной бин спектра является гладкой взвешенной комбинацией линейных локальных моделей, а нелинейность задаётся системой нечетких правил.

4 Двухэтапное обучение

4.1 Этап 1: базовая ANFIS-модель

На первом этапе обучается базовая модель без SHAP-штрафов. Если обозначить обучающий набор первого этапа через

$$\mathcal{D}_{\text{base}} = \{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^{N_{\text{base}}}, \quad (12)$$

то решается задача

$$\theta^{(0)} = \arg \min_{\theta} \mathcal{L}_{\text{main}}(\theta; \mathcal{D}_{\text{base}}). \quad (13)$$

В коде этот этап реализован через `BioAnfisRegressor` с глобальной оптимизацией `OriginalPSO`; в основном конфиге используются 150 эпох PSO и популяция 100 частиц.

4.2 Этап 2: SHAP + Tikhonov донастройка

На втором этапе параметры инициализируются значением $\theta^{(0)}$, после чего модель дообучается на реальных данных. Пусть

$$\mathcal{D}_{\text{real}} = \{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^{N_{\text{real}}} \quad (14)$$

— набор реальных измерений. В текущем основном пайплайне он делится в отношении

$$60\% : 20\% : 20\% \quad (15)$$

на подмножества

$$\mathcal{D}_{\text{real}}^{\text{train}}, \quad \mathcal{D}_{\text{real}}^{\text{val}}, \quad \mathcal{D}_{\text{real}}^{\text{test}}. \quad (16)$$

Для набора из 375 образцов это соответствует разбиению 225/75/75. Именно на финальных 75 реальных образцах считаются основные метрики качества.

5 Основная функция ошибки восстановления

Для мини-батча

$$\mathcal{B} = \{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^B \quad (17)$$

основная ошибка восстановления спектра задаётся как средняя квадратичная ошибка:

$$\mathcal{L}_{\text{main}} = \frac{1}{Bm} \sum_{i=1}^B \|f_{\theta}(\mathbf{x}_i) - \mathbf{y}_i\|_2^2. \quad (18)$$

Здесь множитель $1/m$ появляется потому, что в реализации используется `torch.nn.MSELoss()` с усреднением по всем элементам тензора предсказаний. Если расписать по выходным бинам, то

$$\mathcal{L}_{\text{main}} = \frac{1}{Bm} \sum_{i=1}^B \sum_{k=1}^m (\hat{y}_{ik} - y_{ik})^2. \quad (19)$$

Эта величина является главной задачей оптимизации и остаётся в итоговом функционале на обоих этапах.

6 Градиентная проху-оценка важности признаков

Во второй стадии используется дифференцируемая проху-аппроксимация глобальной важности входных признаков. Для батча предсказаний

$$\hat{\mathbf{Y}} = f_{\theta}(\mathbf{X}) \in \mathbb{R}^{B \times m} \quad (20)$$

сначала вводится скаляризованный выход для каждого образца:

$$s_i = \frac{1}{m} \sum_{k=1}^m \hat{y}_{ik}. \quad (21)$$

Далее для каждого объекта и признака вычисляется градиент

$$g_{ij} = \frac{\partial s_i}{\partial x_{ij}}. \quad (22)$$

Пер-образцовая важность признака определяется как

$$u_{ij} = |g_{ij}| |x_{ij}|. \quad (23)$$

После усреднения по батчу получаем глобальную важность на уровне батча:

$$v_j = \frac{1}{B} \sum_{i=1}^B u_{ij}. \quad (24)$$

Чтобы избежать вырожденных нулевых координат, в реализации применяется отсечение снизу:

$$\tilde{v}_j = \max\left(v_j, 10^{-6} \max_{\ell} v_{\ell}\right). \quad (25)$$

Затем выполняется L_1 -нормировка:

$$p_j = \frac{\tilde{v}_j}{\sum_{\ell=1}^n \tilde{v}_{\ell} + \varepsilon}, \quad \mathbf{p} = (p_1, \dots, p_n)^{\top}. \quad (26)$$

Вектор $\mathbf{p}$ и является основной дифференцируемой проху-оценкой глобальной важности признаков, с которой работают все SHAP-компоненты.

7 Аппроксимация “истинной” SHAP-важности в коде

Согласованность в проекте вычисляется не через полный перебор коалиций и не через независимые SHAP-значения для каждого образца, а через более дешёвую аппроксимацию на уровне батча.

Для текущего батча вводится средний образец

$$\bar{\mathbf{x}} = \frac{1}{B} \sum_{i=1}^B \mathbf{x}_i. \quad (27)$$

В качестве базового вектора используется средний вектор по SHAP-обучающей выборке:

$$\mathbf{b} = \frac{1}{N_{\text{train}}} \sum_{i=1}^{N_{\text{train}}} \mathbf{x}_i. \quad (28)$$

Для каждого признака j строится маскированный вектор

$$\bar{\mathbf{x}}^{(j \leftarrow b_j)} = (\bar{x}_1, \dots, \bar{x}_{j-1}, b_j, \bar{x}_{j+1}, \dots, \bar{x}_n)^\top. \quad (29)$$

Пусть

$$\bar{f}(\mathbf{x}) = \frac{1}{m} \sum_{k=1}^m f_{\theta,k}(\mathbf{x}) \quad (30)$$

— средний по энергетическим бинам отклик модели. Тогда маскирующая важность на уровне батча определяется как

$$\psi_j = \left| \bar{f}(\bar{\mathbf{x}}) - \bar{f}(\bar{\mathbf{x}}^{(j \leftarrow b_j)}) \right|. \quad (31)$$

После этого выполняется нормировка:

$$q_j = \frac{\max(\psi_j, 0)}{\sum_{\ell=1}^n \max(\psi_\ell, 0) + \varepsilon}, \quad \mathbf{q} = (q_1, \dots, q_n)^\top. \quad (32)$$

Если сумма $\sum_j \psi_j$ оказывается вырожденной, в коде используется запасной равномерный вектор $q_j = 1/n$.

Важно, что $\mathbf{q}$ не пересчитывается на каждом батче заново: при `use_true_shap=true` он кэшируется и обновляется раз в K батчей, где по умолчанию $K = 10$.

8 Полная SHAP-регуляризация второй стадии

Итоговый SHAP-штраф состоит из четырёх компонент:

$$\mathcal{L}_{\text{SHAP}} = w_{\text{cons}} R_{\text{cons}} + w_{\text{spar}} R_{\text{spar}} + w_{\text{faith}} R_{\text{faith}} + w_{\text{stab}} R_{\text{stab}}. \quad (33)$$

В основной версии веса w . вычисляются адаптивно. Ниже подробно расписана каждая компонента.

8.1 Компонента согласованности

Согласованность сравнивает дифференцируемую грубую-важность $\mathbf{p}$ с маскирующей аппроксимацией $\mathbf{q}$. Используются два члена: среднеквадратическое отклонение и дивергенция Йенсена–Шеннона.

Сначала вводится

$$\text{MSE}(\mathbf{p}, \mathbf{q}) = \frac{1}{n} \sum_{j=1}^n (p_j - q_j)^2. \quad (34)$$

Далее

$$\mathbf{m} = \frac{1}{2}(\mathbf{p} + \mathbf{q}), \quad (35)$$

$$\text{KL}(\mathbf{p} \parallel \mathbf{m}) = \sum_{j=1}^n p_j \log \frac{p_j}{m_j + \varepsilon}, \quad \text{KL}(\mathbf{q} \parallel \mathbf{m}) = \sum_{j=1}^n q_j \log \frac{q_j}{m_j + \varepsilon}, \quad (36)$$

и

$$\text{JS}(\mathbf{p} \parallel \mathbf{q}) = \frac{1}{2} \text{KL}(\mathbf{p} \parallel \mathbf{m}) + \frac{1}{2} \text{KL}(\mathbf{q} \parallel \mathbf{m}). \quad (37)$$

Тогда согласованность в точной реализации равна

$$R_{\text{cons}} = \frac{\text{MSE}(\mathbf{p}, \mathbf{q}) + \lambda_{\text{JS}} \text{JS}(\mathbf{p} \parallel \mathbf{q})}{1 + \mathcal{L}_{\text{main}}}, \quad \lambda_{\text{JS}} = 0.5. \quad (38)$$

Деление на $1 + \mathcal{L}_{\text{main}}$ реализует precision-aware scaling: когда модель ещё неточна, давление интерпретируемости ослабляется.

8.2 Компонента разреженности

Главная идея — поощрять концентрацию важности на небольшом числе детекторов, не сводя распределение к жёсткому one-hot режиму.

Нормированная энтропия важности записывается как

$$H(\mathbf{p}) = -\frac{1}{\log n} \sum_{j=1}^n p_j \log(p_j + \varepsilon). \quad (39)$$

Также используется коэффициент Джини. Если $p_{(1)} \leq \dots \leq p_{(n)}$ — компоненты $\mathbf{p}$ после сортировки по возрастанию, то

$$G(\mathbf{p}) = \frac{2 \sum_{j=1}^n j p_{(j)}}{n \sum_{j=1}^n p_{(j)} + \varepsilon} - \frac{n+1}{n}. \quad (40)$$

Пусть $G_\star$ — целевой уровень концентрации. В основной реализации по умолчанию используется

$$G_\star = 0.3. \quad (41)$$

Штраф на недостаточную концентрацию:

$$R_{\text{gini}} = \max(0, G_\star - G(\mathbf{p}))^2. \quad (42)$$

Далее строится базовый штраф разреженности

$$R_{\text{spar}}^{\text{base}} = H(\mathbf{p}) + R_{\text{gini}}. \quad (43)$$

Итоговая формула с precision-aware scaling имеет вид

$$R_{\text{spar}} = R_{\text{spar}}^{\text{base}} \left(\rho \frac{1}{1 + \mathcal{L}_{\text{main}}} + (1 - \rho) \right), \quad \rho = 0.7. \quad (44)$$

То есть при хорошем качестве модели штраф на энтропию и недостающий Джини действует сильнее, а при плохом — мягче.

8.3 Компонента достоверности объяснений

В данной реализации faithfulness считается не по полному многомерному вектору выхода, а по его скаляризации через среднее по энергетическим бинам. Это важно, потому что именно такая формула используется в коде.

Для каждого объекта батча определим

$$s_i = \frac{1}{m} \sum_{k=1}^m \hat{y}_{ik}, \quad s_i^{(0)} = \frac{1}{m} \sum_{k=1}^m f_{\theta,k}(\mathbf{0}). \quad (45)$$

Здесь базовая точка выбрана нулевым вектором

$$\mathbf{x}^{(0)} = \mathbf{0} \in \mathbb{R}^n. \quad (46)$$

Градиент уже был определён как $g_{ij} = \partial s_i / \partial x_{ij}$. Тогда первая вариационная аппроксимация изменения отклика имеет вид

$$a_i = \sum_{j=1}^n g_{ij}(x_{ij} - x_j^{(0)}) = \sum_{j=1}^n g_{ij}x_{ij}. \quad (47)$$

Штраф достоверности объяснений вычисляется по формуле

$$R_{\text{faith}} = \frac{1}{1 + \mathcal{L}_{\text{main}}} \cdot \frac{1}{B} \sum_{i=1}^B \left((s_i - s_i^{(0)}) - a_i \right)^2. \quad (48)$$

Следовательно, этот член минимизирует расхождение между реальным изменением среднего отклика модели и его линейной градиентной аппроксимацией относительно нулевой базовой точки.

8.4 Компонента стабильности

В текущем тренере стабильность задаётся не через добавление шума к входам внутри функционала потерь, а через дисперсию важностей внутри батча.

Напомним, что

$$u_{ij} = |g_{ij}| |x_{ij}|. \quad (49)$$

Средняя карта важности по батчу:

$$\bar{u}_j = \frac{1}{B} \sum_{i=1}^B u_{ij}. \quad (50)$$

Тогда стабильность определяется как

$$R_{\text{stab}} = \frac{1}{1 + \mathcal{L}_{\text{main}}} \cdot \frac{1}{Bn} \sum_{i=1}^B \sum_{j=1}^n (u_{ij} - \bar{u}_j)^2. \quad (51)$$

Таким образом, модель поощряется к тому, чтобы образцы внутри батча имели согласованную структуру объяснений по каналам детекторов.

8.5 Адаптивные веса компонент SHAP

Если включён режим `use_adaptive_weights=true` (а в основной конфигурации именно так), веса четырёх компонент пересчитываются динамически на каждом батче.

Пусть

$$\ell = (R_{\text{cons}}, R_{\text{spar}}, R_{\text{faith}}, R_{\text{stab}}). \quad (52)$$

Сначала строятся обратные веса по величине потерь:

$$\tilde{w}_j = \frac{1}{\ell_j + \varepsilon}, \quad w_j^{(0)} = \frac{\tilde{w}_j}{\sum_r \tilde{w}_r}. \quad (53)$$

Далее, если основная ошибка ещё велика, код дополнительно ослабляет *faithfulness* и *stability*. При целевом пороге

$$\tau_{\text{main}} = 0.02 \quad (54)$$

и текущем значении $\mathcal{L}_{\text{main}} > \tau_{\text{main}}$ используется множитель

$$\xi = \frac{\tau_{\text{main}}}{\mathcal{L}_{\text{main}} + \varepsilon}. \quad (55)$$

Тогда веса модифицируются как

$$(w_{\text{cons}}, w_{\text{spar}}, w_{\text{faith}}, w_{\text{stab}}) \leftarrow \text{Norm}(w_{\text{cons}}^{(0)}, w_{\text{spar}}^{(0)}, \xi w_{\text{faith}}^{(0)}, \xi w_{\text{stab}}^{(0)}), \quad (56)$$

где Norm означает повторную нормировку до суммы 1.

Если же `use_adaptive_weights=false`, то используются фиксированные веса из конфигурации:

$$(\gamma_{\text{cons}}, \gamma_{\text{spar}}, \gamma_{\text{faith}}, \gamma_{\text{stab}}) = (0.2, 0.7, 0.05, 0.05) \quad (57)$$

с последующей нормировкой до единичной суммы. Однако для основной версии это запасной режим; в реальном финальном запуске активны именно адаптивные веса.

9 Адаптивная балансировка SHAP и основной задачи

Даже после агрегации компонент $\mathcal{L}_{\text{SHAP}}$ дополнительно нормируется, чтобы интерпретируемость не подавляла основную задачу восстановления спектра.

9.1 ЕМА основной ошибки

В коде поддерживается экспоненциальное скользящее среднее основной ошибки:

$$M_t = \alpha M_{t-1} + (1 - \alpha) \mathcal{L}_{\text{main}}^{(t)}, \quad \alpha = 0.9. \quad (58)$$

На первом шаге берётся $M_0 = \mathcal{L}_{\text{main}}^{(0)}$.

9.2 Динамическое целевое отношение SHAP/main

Пусть $p_t \in [0, 1]$ — относительный прогресс по эпохам:

$$p_t = \frac{t}{T}, \quad (59)$$

где T — общее число эпох второй стадии.

В коде целевое отношение SHAP/main выбирается как

$$\eta_t = \eta_0(0.5 + 0.5p_t), \quad \eta_0 = 0.4. \quad (60)$$

Таким образом, на ранних эпохах SHAP действует мягче, а к концу донастройки его допустимая доля в общей оптимизации увеличивается.

9.3 Нормировка SHAP-штрафа

Пусть

$$r_t = \frac{\mathcal{L}_{\text{SHAP}}^{(t)}}{M_t + \varepsilon} \quad (61)$$

— текущее отношение SHAP-штрафа к ЕМА основной ошибки.

Тогда в точной реализации используется масштабирование

$$s_t = \begin{cases} \frac{\eta_t}{r_t}, & r_t > 2\eta_t, \\ \frac{\eta_t}{r_t}, & r_t < \eta_t/2, \\ 1, & \eta_t/2 \leq r_t \leq 2\eta_t. \end{cases} \quad (62)$$

То есть при слишком большом или слишком маленьком отношении SHAP к основной ошибке штраф нормируется к целевому диапазону.

9.4 Замедление при отсутствии улучшения

В тренере присутствует дополнительный множитель плавной сходимости. Если относительное улучшение основной ошибки становится меньше 0.1%, счётчик отсутствия улучшения увеличивается:

$$q_t = q_{t-1} + 1. \quad (63)$$

Тогда множитель замедления задаётся как

$$c_t = \max\left(\frac{1}{1 + 0.1q_t}, c_{\min}\right), \quad c_{\min} = 0.25. \quad (64)$$

При заметном улучшении основной ошибки счётчик сбрасывается. Нормированный SHAP-штраф имеет вид

$$\tilde{\mathcal{L}}_{\text{SHAP}}^{(t)} = c_t s_t \mathcal{L}_{\text{SHAP}}^{(t)}. \quad (65)$$

9.5 Расписание коэффициента `gamma_t`

Вес SHAP-регуляризации во внешнем функционале также изменяется в процессе донастройки. В коде используется кусочно-линейное расписание с параметрами

$$\gamma_{\text{start}} = 0.02, \quad \gamma_{\text{end}} = 0.10, \quad r_{\text{warm}} = 0.5. \quad (66)$$

Если $p_t = t/T$, то формула в текущей реализации имеет вид

$$\gamma_t = \begin{cases} \gamma_{\text{start}} + (\gamma_{\text{end}} - \gamma_{\text{start}}) \frac{p_t}{r_{\text{warm}}}, & p_t < r_{\text{warm}}, \\ \gamma_{\text{end}}, & p_t \geq r_{\text{warm}}. \end{cases} \quad (67)$$

На практике это означает мягкий линейный старт SHAP-регуляризации на первых $100r_{\text{warm}}\%$ эпох и последующее удержание веса на уровне γ_{end} .

10 Тихоновская регуляризация гладкости спектра

Главная версия проекта дополнительно использует тихоновскую регуляризацию по выходному спектру. Она вводится непосредственно на предсказаниях модели и штрафует резкие осцилляции между соседними энергетическими бинами.

Для одного предсказанного спектра

$$\hat{\mathbf{y}}_i = (\hat{y}_{i1}, \dots, \hat{y}_{im})^\top \quad (68)$$

определяются конечно-разностные операторы:

$$(D_1 \hat{\mathbf{y}}_i)_k = \hat{y}_{i,k+1} - \hat{y}_{i,k}, \quad k = 1, \dots, m-1, \quad (69)$$

$$(D_2 \hat{\mathbf{y}}_i)_k = \hat{y}_{i,k+2} - 2\hat{y}_{i,k+1} + \hat{y}_{i,k}, \quad k = 1, \dots, m-2. \quad (70)$$

В проекте поддерживаются оба порядка, но основная рабочая версия использует второй порядок:

$$p_{\text{Tikh}} = 2. \quad (71)$$

Соответствующий штраф по батчу записывается как

$$R_{\text{Tikh}} = \frac{1}{B(m-2)} \sum_{i=1}^B \sum_{k=1}^{m-2} (\hat{y}_{i,k+2} - 2\hat{y}_{i,k+1} + \hat{y}_{i,k})^2. \quad (72)$$

Именно эта формула соответствует реализации `mean(diffs**2)` в `shap_trainer_improved.py`. Второй порядок особенно важен для спектров, поскольку он штрафует не просто наклон, а локальную кривизну, то есть подавляет искусственные “зубцы” в восстановленном распределении.

11 Итоговый функционал второй стадии

После объединения всех членов вторая стадия минимизирует функционал

$$\mathcal{L}_{\text{total}}^{(t)} = \mathcal{L}_{\text{main}}^{(t)} + \gamma_t \tilde{\mathcal{L}}_{\text{SHAP}}^{(t)} + \lambda_{\text{Tikh}} R_{\text{Tikh}}^{(t)}. \quad (73)$$

В основной конфигурации

$$\lambda_{\text{Tikh}} = 10^{-3}, \quad p_{\text{Tikh}} = 2. \quad (74)$$

Следовательно, ключевая версия метода представляет собой именно задачу

$$\theta^* = \arg \min_{\theta} \left[\mathcal{L}_{\text{main}} + \gamma_t \tilde{\mathcal{L}}_{\text{SHAP}} + 10^{-3} R_{\text{Tikh}} \right], \quad (75)$$

где и интерпретируемость, и гладкость спектра являются мягкими регуляризаторами относительно основной задачи восстановления.

12 Инференс

После обучения модель используется как детерминированный оператор реконструкции:

$$\hat{\mathbf{y}} = f_{\theta^*}(\mathbf{x}). \quad (76)$$

Если данные были нормализованы по сумме, выполняется денормализация:

$$\hat{\mathbf{y}}_{\text{denorm}} = S \hat{\mathbf{y}}'. \quad (77)$$

В проекте дополнительно сохраняются массивы предсказаний, модель, важности признаков, история SHAP-компонент и графические артефакты для последующего анализа.

13 Monte Carlo-оценка неопределённости

Отдельный модуль проекта оценивает чувствительность прогноза к ошибкам во входных измерениях. Для заданного уровня относительной ошибки δ строятся возмущённые входы

$$\mathbf{x}^{(t)} = \mathbf{x} + \boldsymbol{\xi}^{(t)}, \quad \xi_j^{(t)} \sim \mathcal{N}(0, \sigma_j^2), \quad \sigma_j = \frac{\delta}{100} |x_j|. \quad (78)$$

В коде дополнительно выполняется отсечение отрицательных значений:

$$x_j^{(t)} \leftarrow \max(x_j^{(t)}, 0). \quad (79)$$

Если включена нормализация по сумме, то после внесения шума каждое возмущённое наблюдение снова нормируется:

$$\mathbf{x}^{(t)} \leftarrow \frac{\mathbf{x}^{(t)}}{\sum_j x_j^{(t)}}. \quad (80)$$

После T прогонов Monte Carlo получаем ансамбль прогнозов

$$\hat{\mathbf{y}}^{(t)} = f_{\theta^*}(\mathbf{x}^{(t)}), \quad t = 1, \dots, T. \quad (81)$$

По нему вычисляются:

$$\bar{\mathbf{y}} = \frac{1}{T} \sum_{t=1}^T \hat{\mathbf{y}}^{(t)}, \quad (82)$$

$$\text{Std}(\mathbf{y}) = \sqrt{\frac{1}{T} \sum_{t=1}^T (\hat{\mathbf{y}}^{(t)} - \bar{\mathbf{y}})^2}, \quad (83)$$

$$\mathbf{y}^{(p)} = \text{Percentile}_p(\hat{\mathbf{y}}^{(1)}, \dots, \hat{\mathbf{y}}^{(T)}), \quad p \in \{5, 25, 50, 75, 95\}. \quad (84)$$

Тем самым получают среднее предсказание, стандартное отклонение, доверительные интервалы и распределение неопределённости по каждому энергетическому бину.

14 Метрики качества

Для финальной оценки на тестовом наборе используются следующие метрики.

14.1 MSE, RMSE и MAE

Для тестового множества из N образцов:

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N \frac{1}{m} \sum_{k=1}^m (\hat{y}_{ik} - y_{ik})^2, \quad (85)$$

$$\text{RMSE} = \sqrt{\text{MSE}}, \quad (86)$$

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N \frac{1}{m} \sum_{k=1}^m |\hat{y}_{ik} - y_{ik}|. \quad (87)$$

14.2 Взвешенный и средний R^2 по бинам

Поскольку задача многовыходная, в коде сначала считается отдельный коэффициент детерминации для каждого энергетического бина с ненулевой дисперсией:

$$R_k^2 = 1 - \frac{\sum_{i=1}^N (y_{ik} - \hat{y}_{ik})^2}{\sum_{i=1}^N (y_{ik} - \bar{y}_k)^2}, \quad \bar{y}_k = \frac{1}{N} \sum_{i=1}^N y_{ik}. \quad (88)$$

Далее вводятся веса, пропорциональные дисперсии бина:

$$\sigma_k^2 = \text{Var}(y_{1k}, \dots, y_{Nk}), \quad \omega_k = \frac{\sigma_k^2}{\sum_{\ell} \sigma_{\ell}^2}. \quad (89)$$

Тогда основная итоговая метрика в проекте — variance-weighted R^2 :

$$R_{\text{weighted}}^2 = \sum_k \omega_k R_k^2. \quad (90)$$

Дополнительно сохраняется невзвешенное среднее по выходам:

$$R_{\text{mean}}^2 = \frac{1}{m'} \sum_{k=1}^{m'} R_k^2, \quad (91)$$

где m' — число бинов с ненулевой дисперсией.

14.3 Метрики по диапазонам энергий

Для более детального анализа 60 бинов делятся на три диапазона:

$$[0, 19], \quad [20, 39], \quad [40, 59]. \quad (92)$$

Для каждого диапазона отдельно вычисляются MSE, RMSE, MAE и R^2 .

15 Параметры основной Tikhonov-версии

Финальная рабочая конфигурация метода задаётся следующими параметрами:

- число правил ANFIS: $R = 50$;
- функции принадлежности: Gaussian;
- этап 1 (PSO): 150 эпох, популяция 100;
- этап 2 (донастройка): 300 эпох, размер батча 32, $\text{lr} = 10^{-3}$;
- расписание SHAP-веса: $\gamma_{\text{start}} = 0.02$, $\gamma_{\text{end}} = 0.10$;
- warmup-доля для γ_t : $r_{\text{warm}} = 0.5$;
- adaptive gamma: включён;
- целевое отношение SHAP/main: $\eta_0 = 0.40$;
- минимальный множитель плавной сходимости: $c_{\text{min}} = 0.25$;
- adaptive component weights: включены;
- true-SHAP reference: включён;
- активные SHAP-компоненты: *consistency*, *sparsity*, *faithfulness*, *stability*;
- резервные фиксированные веса компонент: $(0.2, 0.7, 0.05, 0.05)$ для *consistency*, *sparsity*, *faithfulness*, *stability*;
- тихоновская регуляризация: $\lambda_{\text{Tikh}} = 2 \cdot 10^{-3}$, порядок $p_{\text{Tikh}} = 2$.

16 Практическая интерпретация метода

С математической точки зрения итоговый метод сочетает три уровня априорной информации:

1. **Аппроксимация данных:** член $\mathcal{L}_{\text{main}}$ заставляет модель правильно восстанавливать спектр.
2. **Интерпретируемость:** SHAP-компоненты добиваются того, чтобы карта важности детекторов была согласованной, разреженной, достоверной и устойчивой.
3. **Физическая гладкость спектра:** тихоновский член подавляет нефизичные локальные осцилляции по энергии.

По сути, оптимизация решает компромисс между точностью, объяснимостью и гладкостью, причём компромисс настраивается автоматически по ходу обучения через γ_t , ЕМА-нормировку и адаптивные веса компонент SHAP.

17 Экспериментальные результаты для основной версии

Для основного запуска `tikhonov_stronger_20260319` на финальном реальном тесте получены следующие метрики:

$$\text{MSE} = 0.01060296, \quad (93)$$

$$\text{RMSE} = 0.10297068, \quad (94)$$

$$\text{MAE} = 0.04890357, \quad (95)$$

$$R_{\text{weighted}}^2 = 0.83702629, \quad (96)$$

$$R_{\text{mean}}^2 = 0.55409377. \quad (97)$$

Метрики по диапазонам энергий:

- бины $[0, 19]$: $R_{\text{weighted}}^2 = 0.95486219$;
- бины $[20, 39]$: $R_{\text{weighted}}^2 = 0.75988831$;
- бины $[40, 59]$: $R_{\text{weighted}}^2 = 0.68916792$.

Нормализованная глобальная SHAP-важность в этом запуске образует распределение с единичной суммой. Пять наиболее значимых детекторов имеют веса

$$Q3 = 0.269309, \quad Q5 = 0.206911, \quad Q6 = 0.137873, \quad Q4 = 0.095738, \quad Q1 = 0.078220. \quad (98)$$

Это подтверждает, что модель не размывает вклад равномерно по всем 10 каналам, а выделяет устойчивое ядро информативных детекторов.

Средний вклад регуляризаторов в итоговый функционал для этого запуска составил

$$\mathbb{E} [\gamma_t \tilde{\mathcal{L}}_{\text{SHAP}}^{(t)}] = 5.54 \cdot 10^{-5}, \quad \mathbb{E} [\lambda_{\text{Tikh}} \mathcal{L}_{\text{Tikh}}^{(t)}] = 4.23 \cdot 10^{-4}, \quad (99)$$

а средняя доля регуляризации в полном функционале достигла

$$\mathbb{E} \left[\frac{\gamma_t \tilde{\mathcal{L}}_{\text{SHAP}}^{(t)} + \lambda_{\text{Tikh}} \mathcal{L}_{\text{Tikh}}^{(t)}}{\mathcal{L}_{\text{total}}^{(t)}} \right] = 0.0653. \quad (100)$$

По сравнению с предыдущим более мягким режимом вклад SHAP во внешний функционал вырос примерно в 29.8 раза, а вклад тихоновского члена — примерно в 2.0 раза, при этом итоговые метрики точности не ухудшились.

Дополнительный абляционный анализ на уменьшенной конфигурации показал:

- удаление *sparsity* даёт наибольшую просадку по R^2_{weighted} , значит этот компонент является главным носителем полезной SHAP-структуры;
- удаление *consistency* почти не ухудшает ассигасу, но делает глобальную SHAP-карту более концентрированной и менее энтропийной;
- *faithfulness* и *stability* выступают как поддерживающие компоненты, влияющие слабее, но стабилизирующие итоговую структуру регуляризации.

При этом адаптивная схема весов выравнивает не сами коэффициенты, а *эффективные взвешенные сигналы* компонент, поэтому даже при сильно различающихся raw weight значения *consistency*, *sparsity*, *faithfulness* и *stability* дают сравнимый вклад в SHAP-часть функционала.

Для анализа чувствительности к входной погрешности был выполнен Monte Carlo-анализ. При уровне ошибки входных каналов 0.5% получено:

$$\text{mean_std} = 3.8511 \cdot 10^{-5}, \quad \text{max_std} = 2.1553 \cdot 10^{-4}. \quad (101)$$

При увеличении ошибки до 10% соответствующие значения выросли до

$$\text{mean_std} = 7.5806 \cdot 10^{-4}, \quad \text{max_std} = 4.1703 \cdot 10^{-3}. \quad (102)$$

Следовательно, главная SHAP + Tikhonov версия сохраняет приемлемую устойчивость к реалистичным ошибкам измерений и одновременно даёт физически более гладкие спектры по сравнению с вариантом без регуляризации гладкости.